Requirements: Feedback Intelligence Platform

Status	Approved
Version	1.1
Date	2026-11-05
Owner	Benjamin Rose
Related	Project plan

This document defines what the Feedback Intelligence Platform must do. It contains personas, user stories with acceptance criteria, functional and non-functional requirements, priorities, and a traceability matrix linking every requirement to the user stories it serves.

Conventions

- **IDs** are stable. Code, tests, and pull requests refer to them (for example, "Implements FR-ING-02").
- **Priority** uses MoSCoW: Must, Should, Could, Won't. A functional requirement takes the highest priority among the stories it traces to.
- **Milestones** (M0–M8) are defined in the project plan.

1. Personas

Priyanka, Product Manager *(primary user)*

Owns a developer tool and spends Monday mornings figuring out what to prioritize. She doesn't have time to read hundreds of issues.

- **Wants:** the top problems this week, anything suddenly spiking, and a short summary she can share with leadership.
- **Pain points:** issue lists are noisy, duplicated, and full of logs.

Marcus, Engineering Lead *(secondary user)*

Triages bugs for his team and decides what gets fixed first.

- **Wants:** to filter to critical bugs in his area, see a summary, and jump to the original issue.
- **Pain points:** severity is rarely labeled consistently, so urgent issues get buried.

Operator *(admin)*

The project owner, who keeps the system running and changes its behavior over time.

- **Wants:** to know immediately when something breaks, see pipeline health at a glance, and change repositories, categories, or prompts safely through a pull request.
- **Pain points:** silent failures, and changes that break things without warning.

Elliot, Technical Interviewer *(evaluator)*

Has a few minutes to judge a candidate's engineering ability.

- **Wants:** a live link that clearly works, a README that explains the system quickly, and easy paths into the design and tests.
- **Pain points:** projects that don't run, or that hide how they were built.

2. User stories

ID	Story	Priority	Milestone
US-01	As a product manager, I want an overview of the past 7 days (new issues, bug share, critical count, median time to close) with week-over-week changes, so that I can see the state of my product at a glance.	**Must**	M3
US-02	As a product manager, I want to see trends in issue volume and categories over time, so that I can tell whether things are improving or getting worse.	**Must**	M3
US-03	As a product manager, I want a list of emerging topics that are growing quickly, so that I can catch new problems, such as a bad release, early.	**Should**	M5
US-04	As a product manager, I want a short weekly digest of the top themes, so that I can share an update with leadership without writing it myself.	**Should**	M5
US-05	As an engineering lead, I want every issue to have a category and severity, so that I can prioritize consistently even when the original issue isn't labeled.	**Must**	M4
US-06	As an engineering lead, I want to search and filter issues by repository, category, severity, and date, so that I can find the issues relevant to my team.	**Must**	M3
US-07	As an engineering lead, I want a plain-English summary of each long issue, so that I can understand it without reading pages of logs.	**Must**	M4
US-08	As an engineering lead, I want each issue to link to the original on GitHub, so that I can read the full discussion and act on it.	**Must**	M3
US-09	As the operator, I want the pipeline to run every day without manual steps, so that the dashboard stays current without my attention.	**Must**	M2–M3
US-10	As the operator, I want a GitHub issue opened automatically when the pipeline fails or data is stale, so that I learn about problems right away.	**Must**	M3

ID	Story	Priority	Milestone
US-11	As the operator, I want a pipeline health page showing recent runs, records processed, AI usage, and data freshness, so that I can confirm the system is healthy at a glance.	**Should**	M5
US-12	As the operator, I want to change tracked repositories, categories, and thresholds in a config file through a pull request, so that changes are reviewed, validated, and reversible.	**Must**	M3
US-13	As the operator, I want prompt changes to be checked against the evaluation set in CI, so that a change can't silently make the AI less accurate.	**Must**	M4
US-14	As the operator, I want to trigger a manual run or a backfill for a date range, so that I can recover from failures or reprocess data after improvements.	**Should**	M3–M4
US-15	As the operator, I want to run the whole system locally with one command, so that I can develop and demo it offline.	**Must**	M1, extended M3
US-16	As an interviewer, I want a README with a live dashboard link, a short explanation, and links to the design docs, so that I can understand the project in a few minutes.	**Must**	M1, finished M7
US-17	As an interviewer, I want to see tests, CI results, and the decision records, so that I can judge the engineering process, not just the output.	**Must**	M0 onward
US-18	As someone who filed a public issue, I want my personal information kept out of AI processing and off the dashboard, so that my data isn't spread further than I intended.	**Must**	M2–M3
US-19	As an engineering lead, I want to export filtered issues as CSV, so that I can analyze them in other tools.	**Could**	M8
US-20	As a product manager, I want a Power BI report on the same data, so that I can explore it with tools my organization already uses.	**Could**	M8
US-21	As the operator, I want a local Spark prototype that processes the same data with different partitioning strategies, so that I can compare approaches against SQL Server.	**Could**	M8

Issue authors (US-18) are stakeholders rather than users: they never use the system, but their writing is what it processes.

3. Acceptance criteria

Criteria use short checklists. Scenarios where the starting situation matters use **Given / When / Then**.

US-01: Overview

- Shows, for the past 7 days: new issues, percentage that are bugs, number of critical issues, and median time to close.
- Each figure shows its change from the previous 7 days (up or down, with the amount).
- Can be filtered to one repository or all repositories.
- Shows the date and time the data was last updated.

US-02: Trends

- Shows weekly issue volume over at least the past 12 weeks, or all available history if less.
- Shows the category mix over the same period.
- Can be filtered by repository and date range.

US-03: Emerging topics

- Lists groups of similar issues, ranked by growth over the past 7 days compared to the prior 7 days.
- Each topic shows a short name, a one-paragraph summary, its issue count, and its growth.
- Selecting a topic shows the issues in it, each linking to GitHub.
- A topic needs at least 3 issues to appear.

US-04: Weekly digest

- A new digest is generated once a week, covering the previous 7 days.
- Contains the top 3 themes, anything new or spiking, and anything improving.
- Under 300 words.
- Past digests stay available in an archive.

US-05: Category and severity

- Every processed issue has exactly one category: bug, feature request, question, docs, or performance.
- Every processed issue has one severity: low, medium, high, or critical, with a one-line reason.
- Issues the AI can't classify are marked "unclassified" rather than guessed.
- Labels are marked as AI-generated.

US-06: Search and filter

- Filter by repository, category, severity, and date range, in any combination.
- Search by text in the title or summary.
- Results appear in under 2 seconds for the full dataset.

US-07: Summaries

- Every processed issue longer than 500 characters has a summary of 1–3 sentences.
- Summaries never contain personal information.

US-08: Link to original

- Every issue in the dashboard links to its original GitHub URL.
- The link opens in a new tab.

US-09: Daily automated run

- The pipeline starts automatically once a day at 06:00 UTC.
- A full run needs no manual steps.
- Only issues created or updated since the last successful run are fetched.
- **Given** a run fails, **when** the next scheduled run starts, **then** it fetches everything since the last *successful* run, so no data is skipped.

US-10: Automatic alerts

- **Given** any pipeline step fails, **when** the run ends, **then** a GitHub issue is opened with the failed step, the error message, and a link to the run logs.
- **Given** data is more than 48 hours old, **when** the pipeline or a freshness check runs, **then** a stale-data alert issue is opened.
- **Given** an alert issue for the same problem is already open, **when** the problem repeats, **then** a comment is added to that issue instead of opening a duplicate.
- **Given** a data-quality check fails, **when** the run ends, **then** nothing is published and the dashboard keeps showing the last good data.

US-11: Pipeline health page

- Shows the last 30 runs with date, status, and duration.
- For each run: issues fetched, issues processed by AI, AI requests used, and cache hits.
- Shows current data freshness (time since last successful update).
- Shows current AI usage against the daily free limit.

US-12: Config through pull requests

- Tracked repositories, categories, severity levels, and thresholds live in one config file in the repository.
- **Given** a pull request changes the config, **when** CI runs, **then** the config is validated, and an invalid config fails the check.

- A config change takes effect on the next run after merge, with no code changes needed.

US-13: Prompt evaluation gate

- Prompts live in versioned files in the repository.
- **Given** a pull request changes a prompt, **when** CI runs, **then** the evaluation set is scored and the accuracy is reported on the pull request.
- **Given** accuracy falls below 80%, **when** CI finishes, **then** the check fails and the change can't be merged.

US-14: Manual runs and backfills

- The pipeline can be started manually from the GitHub Actions tab.
- A manual run can take an optional start and end date to backfill that range.
- **Given** a backfill would exceed the daily AI limit, **when** it runs, **then** it processes as much as the limit allows, newest first, and resumes on the next run.
- Reprocessing never creates duplicate issues.

US-15: Local run

- One command (`docker compose up`) starts SQL Server, Presidio, and any other local services.
- The pipeline can run locally against them with documented steps.
- Local runs never publish to the live dashboard.

US-16: README

- The README is the first thing on the repository page and links to the live dashboard at the top.
- It explains what the system does and who it's for in a few sentences.
- It includes an architecture diagram, or links to one.
- It links to the plan, requirements, design documents, and decision records.
- It includes local setup steps that a newcomer can follow in under 30 minutes.

US-17: Visible engineering process

- The README shows a CI status badge.
- Test coverage is reported on each pull request.
- Every pull request uses the template, with the self-review checklist and AI-disclosure section filled in.
- Architecture Decision Records live in `docs/adr/`, one file per decision.

US-18: Privacy for issue authors

- **Given** an issue contains an email address, IP address, access token, or a name in a file path, **when** it's processed, **then** those values are replaced with placeholders before any AI call or publishing.
- GitHub usernames are not stored or shown anywhere on the dashboard.
- A test set of issues with known PII must pass with zero leaks before every deployment.
- **Given** PII is found in published data, **when** it's reported, **then** it's removed and the dashboard republished, and the missed pattern gets a new test.

US-19: CSV export

- The issue explorer exports the current filtered results as a CSV file.
- The export contains only published (redacted) fields.

US-20: Power BI report

- A Power BI Desktop report file in the repository loads the exported data.
- It includes at least Overview and Trends pages.
- Screenshots appear in the README, since publishing online requires a paid license.

US-21: Spark prototype

- PySpark runs locally in Docker.
- It compares at least two partitioning strategies on the same data.
- Results and conclusions are written up in `docs/experiments/`.

4. Functional requirements

Ingestion (ING)

ID	Requirement	Traces to	Priority	Milestone
FR-ING-01	The system shall fetch issues from each repository listed in the config file using the GitHub API.	US-09, US-12	**Must**	M2
FR-ING-02	The system shall fetch only issues created or updated since the last successful run, tracked per repository. On the first run for a repository, it shall fetch issues within a history window set in the config file.	US-02, US-09	**Must**	M3
FR-ING-03	The system shall exclude pull requests, which the GitHub API returns alongside issues.	US-01, US-02, US-06	**Must**	M2

ID	Requirement	Traces to	Priority	Milestone
FR-ING-04	The system shall respect GitHub API rate limits, waiting and retrying when a limit is reached.	US-09	**Must**	M3
FR-ING-05	The system shall validate each fetched record against an expected schema and reject malformed records.	US-10	**Must**	M3
FR-ING-06	The system shall support a manual run with an optional start and end date for backfills.	US-14	**Should**	M3

Privacy (PRV)

ID	Requirement	Traces to	Priority	Milestone
FR-PRV-01	The system shall redact emails, IP addresses, access tokens, and personal names in file paths from issue text before storage in cleaned tables, AI processing, or publishing.	US-07, US-18	**Must**	M3
FR-PRV-02	The system shall not store or publish GitHub usernames.	US-18	**Must**	M2
FR-PRV-03	The system shall keep raw, unredacted text only in the temporary pipeline database, never in committed files or published output.	US-18	**Must**	M2
FR-PRV-04	CI shall run a test set of issues with known PII before every deployment and fail if any PII appears in the output.	US-18	**Must**	M3

Data storage and transformation (DAT)

ID	Requirement	Traces to	Priority	Milestone
FR-DAT-01	The system shall apply versioned schema migrations to SQL Server at the start of each run.	US-09	**Must**	M2
FR-DAT-02	The system shall restore prior history into SQL Server at the start of each run.	US-02, US-09	**Must**	M3
FR-DAT-03	The system shall upsert issues so that reprocessing never creates duplicates.	US-09, US-14	**Must**	M3
FR-DAT-04	The system shall transform data through three layers: raw, cleaned, and reporting.	US-01, US-02	**Must**	M3
FR-DAT-05	The system shall run data-quality checks (row counts, required fields, valid values, freshness) before publishing, and stop the run if any fail.	US-10	**Must**	M3
FR-DAT-06	The system shall save updated history at the end of each successful run.	US-02, US-09	**Must**	M3

AI enrichment (AI)

ID	Requirement	Traces to	Priority	Milestone
FR-AI-01	The system shall assign each new or changed issue one category and one severity with a one-line reason.	US-05	**Must**	M4
FR-AI-02	The system shall generate a 1–3 sentence summary for issues longer than 500 characters.	US-07	**Must**	M4
FR-AI-03	The system shall cache AI results by a hash of the redacted text and never re-send text that's already been processed.	US-09, US-14	**Must**	M4
FR-AI-04	The system shall require structured (JSON) AI output, validate it, and mark issues "unclassified" when output is invalid.	US-05	**Must**	M4
FR-AI-05	The system shall track AI requests per run and stop sending requests before exceeding the daily free limit, processing newest issues first.	US-11, US-14	**Must** (required by NFR-COST-02)	M4
FR-AI-06	The system shall load prompts from versioned files in the repository.	US-13	**Must**	M4

Insights (INS)

ID	Requirement	Traces to	Priority	Milestone
FR-INS-01	The system shall compute weekly metrics per repository: issue volume, category mix, critical count, and median time to close.	US-01, US-02	**Must**	M3: volume and median time to close; M4: category mix and critical count
FR-INS-02	The system shall group similar issues into topics and rank them by growth over the past 7 days versus the prior 7 days.	US-03	**Should**	M5
FR-INS-03	The system shall show a topic only if it contains at least 3 issues.	US-03	**Should**	M5
FR-INS-04	The system shall generate a short name and summary for each emerging topic.	US-03	**Should**	M5
FR-INS-05	The system shall generate a weekly digest of under 300 words and keep all past digests.	US-04	**Should**	M5

Dashboard (DSH)

ID	Requirement	Traces to	Priority	Milestone
FR-DSH-01	The system shall publish a static dashboard to GitHub Pages after each successful run.	US-01, US-09	**Must**	M2
FR-DSH-02	The dashboard shall have Overview, Trends, and Issue explorer pages.	US-01, US-02, US-06	**Must**	M3
FR-DSH-03	The dashboard shall let users filter by repository, category, severity, and date range, and search by text, entirely in the browser.	US-01, US-02, US-06	**Must**	M3: repository, date range, and text; M4: category and severity
FR-DSH-04	The dashboard shall link every issue to its original GitHub URL, opening in a new tab.	US-03, US-08	**Must**	M3
FR-DSH-05	The dashboard shall show when the data was last updated.	US-01, US-11	**Must**	M3
FR-DSH-06	The dashboard shall label AI-generated categories, severities, and summaries as AI-generated.	US-05	**Must**	M4
FR-DSH-07	The dashboard shall have Emerging topics, Weekly digest, and Pipeline health pages.	US-03, US-04, US-11	**Should**	M5
FR-DSH-08	The issue explorer shall export the current filtered results as CSV, using published fields only.	US-19	**Could**	M8

Operations (OPS)

ID	Requirement	Traces to	Priority	Milestone
FR-OPS-01	The system shall run on a daily schedule and support manual runs.	US-09, US-14	**Must**	M2
FR-OPS-02	The system shall open a GitHub issue when a run fails or data is older than 48 hours, with the failed step, error, and log link.	US-10	**Must**	M3
FR-OPS-03	The system shall add a comment to an existing open alert instead of opening a duplicate.	US-10	**Must**	M3
FR-OPS-04	The system shall record metrics for every run: status, duration, issues fetched, issues processed, AI requests used, and cache hits.	US-11	**Should**	M5
FR-OPS-05	The system shall keep the last successfully published data live whenever a run fails.	US-10	**Must**	M3

Configuration (CFG)

ID	Requirement	Traces to	Priority	Milestone
FR-CFG-01	The system shall read tracked repositories, categories, severity levels, history window, and thresholds from a single config file.	US-12	**Must**	M3: repositories, history window, and thresholds; M4: categories and severity levels
FR-CFG-02	The system shall validate the config file in CI and fail the check if it's invalid.	US-12	**Must**	M3

Development and delivery (DEV)

ID	Requirement	Traces to	Priority	Milestone
FR-DEV-01	CI shall build the code, run all tests, check formatting, and scan for secrets on every pull request.	US-17	**Must**	M1
FR-DEV-02	CI shall score the AI evaluation set when prompts change and fail below 80% accuracy.	US-13	**Must**	M4
FR-DEV-03	CI shall report test coverage on every pull request.	US-17	**Must**	M1
FR-DEV-04	The repository shall provide a Docker Compose setup that starts all local services with one command.	US-15	**Must**	M1, extended M3
FR-DEV-05	Local runs shall never publish to the live dashboard.	US-15	**Must**	M1
FR-DEV-06	The repository shall include a README, design documents, and decision records as described in US-16 and US-17.	US-16, US-17	**Must**	M0–M7

Extensions (EXT)

ID	Requirement	Traces to	Priority	Milestone
FR-EXT-01	The repository shall include a Power BI Desktop report built on the exported data.	US-20	**Could**	M8
FR-EXT-02	The repository shall include a local Spark prototype comparing partitioning strategies, with a written comparison.	US-21	**Could**	M8

5. Non-functional requirements

There are 28 non-functional requirements. All are **Must**, because they come directly from the project's constraints and success criteria.

Performance (PERF)

ID	Requirement	Source
NFR-PERF-01	Dashboard filters and search shall return results in under 2 seconds on the full dataset.	US-06
NFR-PERF-02	The dashboard's first page shall load in under 3 seconds on a 10 Mbps connection.	US-16
NFR-PERF-03	A daily pipeline run shall complete in under 30 minutes.	US-09

Reliability (REL)

ID	Requirement	Source
NFR-REL-01	The pipeline shall succeed on at least 95% of days over any 30-day period.	Success criteria
NFR-REL-02	Published data shall never be more than 48 hours old without an open alert explaining why.	Success criteria
NFR-REL-03	A failed run shall never leave the dashboard broken or partially updated.	US-10

Scalability (SCL)

ID	Requirement	Source
NFR-SCL-01	Adding a repository shall require only a config change, with no code changes.	US-12
NFR-SCL-02	The system shall handle all three repositories within the free AI limit by batching and caching.	Risk R2
NFR-SCL-03	Stored data shall stay under 2 GB, well within SQL Server Express's 10 GB limit.	Constraints

Security (SEC)

ID	Requirement	Source
NFR-SEC-01	No secrets shall be committed to the repository; only the built-in `GITHUB_TOKEN` is used.	Risk R11
NFR-SEC-02	Workflows shall use the minimum permissions each step needs (principle of least privilege).	Risk R11
NFR-SEC-03	Issue text shall be treated strictly as data in AI prompts, and AI output shall be validated before use.	Risk R4
NFR-SEC-04	Dependencies shall be scanned for known vulnerabilities in CI.	Best practice

Privacy (PRIV)

ID	Requirement	Source
NFR-PRIV-01	Published data shall contain zero known PII, verified by the PII test set.	Success criteria, US-18
NFR-PRIV-02	Published and stored data shall contain only the fields listed in the data contract, defined in the design phase.	US-18

Cost (COST)

ID	Requirement	Source
NFR-COST-01	Operating cost shall be $0, with no payment method on file with any service.	Goal 2
NFR-COST-02	AI usage shall stay within the free daily limit, with no rate-limit failures over 30 days.	Success criteria

Maintainability and testability (MNT)

ID	Requirement	Source
NFR-MNT-01	Automated test coverage for C# code shall be at least 80%.	Success criteria
NFR-MNT-02	Every SQL transform shall have an expected-output test.	Success criteria
NFR-MNT-03	External services (GitHub API, AI provider, PII detector) shall sit behind interfaces so they can be replaced or faked in tests.	Risk R1
NFR-MNT-04	Code shall pass automated formatting and style checks.	Goal 3

Observability (OBS)

ID	Requirement	Source
NFR-OBS-01	Pipeline logs shall be structured and include the run ID and step name on every entry.	US-10, US-11
NFR-OBS-02	Every failure shall be visible within one run through an alert issue.	US-10

Usability and accessibility (USE)

ID	Requirement	Source
NFR-USE-01	The dashboard shall be usable on desktop and mobile screen widths.	US-01

ID	Requirement	Source
NFR-USE-02	Charts shall not rely on color alone, and text shall meet WCAG AA contrast guidelines.	Best practice
NFR-USE-03	A newcomer shall be able to run the system locally from the README in under 30 minutes.	Success criteria

Compliance (CMP)

ID	Requirement	Source
NFR-CMP-01	SQL Server Developer edition shall never be used in the production pipeline.	Constraints
NFR-CMP-02	Use of GitHub data and services shall follow GitHub's terms of service.	Constraints, assumption A8

6. Prioritization

Method

User stories are prioritized with **MoSCoW**:

- **Must have:** the project fails without it.
- **Should have:** important, but the project still works without it.
- **Could have:** built only after all Must and Should items are complete.
- **Won't have (this time):** explicitly excluded.

Functional requirements take the highest priority among the stories they trace to. All non-functional requirements are Must.

Summary

Priority	User stories	Functional requirements
Must	14	40
Should	4	7
Could	3	3
Total	**21**	**50**

If only the Must stories are built, the project still meets all four goals in the project plan. The Should stories strengthen it, and the Could stories extend it into Power Platform and Spark.

Won't have (this time)

- User accounts or personalized views.
- Writing labels or comments back to GitHub.
- Real-time updates.
- A live AI chat feature.
- Feedback sources other than GitHub issues.

7. Traceability matrix

Every user story and the functional requirements that implement it.

Story	Priority	Milestone	Functional requirements
US-01 Overview	**Must**	M3	FR-ING-03, FR-DAT-04, FR-INS-01, FR-DSH-01, FR-DSH-02, FR-DSH-03, FR-DSH-05
US-02 Trends	**Must**	M3	FR-ING-02, FR-ING-03, FR-DAT-02, FR-DAT-04, FR-DAT-06, FR-INS-01, FR-DSH-02, FR-DSH-03
US-03 Emerging topics	**Should**	M5	FR-INS-02, FR-INS-03, FR-INS-04, FR-DSH-04, FR-DSH-07
US-04 Weekly digest	**Should**	M5	FR-INS-05, FR-DSH-07
US-05 Category and severity	**Must**	M4	FR-AI-01, FR-AI-04, FR-DSH-06
US-06 Search and filter	**Must**	M3	FR-ING-03, FR-DSH-02, FR-DSH-03
US-07 Summaries	**Must**	M4	FR-PRV-01, FR-AI-02
US-08 Link to original	**Must**	M3	FR-DSH-04
US-09 Daily automated run	**Must**	M2–M3	FR-ING-01, FR-ING-02, FR-ING-04, FR-DAT-01, FR-DAT-02, FR-DAT-03, FR-DAT-06, FR-AI-03, FR-DSH-01, FR-OPS-01
US-10 Automatic alerts	**Must**	M3	FR-ING-05, FR-DAT-05, FR-OPS-02, FR-OPS-03, FR-OPS-05
US-11 Pipeline health page	**Should**	M5	FR-AI-05, FR-DSH-05, FR-DSH-07, FR-OPS-04
US-12 Config through PRs	**Must**	M3	FR-ING-01, FR-CFG-01, FR-CFG-02
US-13 Prompt evaluation gate	**Must**	M4	FR-AI-06, FR-DEV-02
US-14 Manual runs and backfills	**Should**	M3–M4	FR-ING-06, FR-DAT-03, FR-AI-03, FR-AI-05, FR-OPS-01

Story	Priority	Milestone	Functional requirements
US-15 Local run	**Must**	M1, extended M3	FR-DEV-04, FR-DEV-05
US-16 README	**Must**	M1, finished M7	FR-DEV-06
US-17 Visible engineering process	**Must**	M0 onward	FR-DEV-01, FR-DEV-03, FR-DEV-06
US-18 Privacy for issue authors	**Must**	M2–M3	FR-PRV-01, FR-PRV-02, FR-PRV-03, FR-PRV-04
US-19 CSV export	**Could**	M8	FR-DSH-08
US-20 Power BI report	**Could**	M8	FR-EXT-01
US-21 Spark prototype	**Could**	M8	FR-EXT-02

Revision history

Version	Date	Change
1.1	2026-11-05	M3 review: split the milestones of FR-INS-01, FR-DSH-03, and FR-CFG-01, whose category and severity parts depend on AI classification in M4. No requirement text changed.
1.0	2026-10-14	Initial requirements approved